Práctica 02.
Aprendiendo Linux.
Introducción al SO Linux



Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera /José Ramón Balsas Almagro
Copyright: Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional (CC BY-NC-ND 4.0)



Introducción (entrada al sistema, salida de sistema, introducción de órdenes, algunas órdenes útiles, el Shell, las llamadas al sistema `fork()`, `exec()` y `wait()`), los metacaracteres y cómo los interpreta el shell, la redirección, la interconexión.

Este guión toma como referencia Ubuntu 24.04 LTS, la versión instalada en el laboratorio. Cuando una orden o una pantalla difieren en Ubuntu 26.04 LTS, la versión que probablemente tenga instalada en su equipo, se indica en un recuadro como los que encontrará más adelante.

Contenidos

Contenidos **1**
1. Entrada al Sistema **3**
2. Salida del sistema **5**
3. Introducción de órdenes **6**
4. Algunas órdenes útiles **9**
5. El Shell **10**
5.1. El shell como ejecutor de órdenes 11
5.2. El shell como intérprete de metacaracteres 15
5.3. Abreviaturas de nombres de ficheros 15
5.4. Redireccionamiento de la E/S 19
5.5. Interconexiones (pipes) 24
Ejercicios **26**
Preguntas frecuentes **27**
¿Por qué al ejecutar una orden en el shell me dice "orden no encontrada" o "command not found"? 27
fork y wait ¿Para qué sirven las llamadas fork y wait? 28
Shell ¿Cómo interpreta las órdenes el shell? 30
Argumentos ¿Cómo se pasan argumentos a una orden? 32

1. Entrada al Sistema



En 1969, Ken Thompson y Dennis Ritchie comenzaron a desarrollar lo que luego se convirtió en el sistema operativo UNIX.

https://www.youtube.com/watch?v=7FjX7r5icV8

- Unix fue desarrollado por los laboratorios Bell y fue operacional en un PDP-7 en 1970.
- El primer hito notable fue portar el sistema UNIX desde el PDP-7 al PDP-11.
- El siguiente hito importante fue la reescritura de UNIX en el lenguaje de programación C.
- Hoy en día, prácticamente todas las implementaciones de UNIX están escritas en C.

El sistema LINUX es un entorno de computación multiproceso, multiusuario e interactivo.

- **Multiproceso** significa que en un instante dado se pueden ejecutar varios procesos o programas a la vez.
- **Multiusuario** significa que muchos usuarios pueden estar usando el sistema a la vez.

En un sistema operativo multiusuario es vital que el sistema pueda **conocer y diferenciar** a los distintos usuarios activos en un instante dado. Eso se consigue mediante la conexión (*login*) y desconexión (*logout* o *logoff*) del sistema. Para que un usuario pueda identificarse en el sistema y ganar acceso al mismo, debe tener una **cuenta de usuario** activa definida en el sistema (las credenciales). Una cuenta de usuario es información sobre el usuario que el sistema almacena y una carpeta en el disco propiedad del usuario. Las **cuentas de usuario** las administra (crea, modifica y elimina) el administrador del sistema, que es un usuario con permisos especiales que se encarga de gestionar el sistema operativo. Cuando un usuario quiere acceder a un sistema operativo multiusuario, el sistema le pide sus **credenciales de acceso**. Las credenciales de acceso son un

- **nombre de usuario** (*login*) y
- **una clave de acceso** o contraseña (*password*).

Si el sistema operativo ha arrancado en modo consola (en systemd, el objetivo multi-user.target)[1], o bien nos hemos conectado a él desde otro ordenador mediante conexión remota, el sistema nos pedirá en la pantalla el nombre de usuario escribiendo el mensaje:

[1] **systemd y multi-user.target:** `systemd` es el administrador del sistema en Linux que gestiona el arranque de servicios. Un *target* (objetivo) es un grupo de servicios que define un estado del sistema. El `multi-user.target` (equivalente al antiguo *Runlevel 3*) arranca el sistema en **modo consola multiusuario con red y servicios activos, pero sin interfaz gráfica**, optimizando el uso de recursos en servidores.

```
[nombre ordenador] login:
```

```
Ubuntu 24.04.5 LTS so-lab01 tty3

so-lab01 login: _
```

Una vez introducido el nombre de usuario hay que pulsar *Enter*, y el sistema pregunta la clave de acceso. Hay que teclearla y pulsar *Enter*. Mientras se teclea la clave, ésta no se muestra en pantalla para evitar que alguien pueda verla y acceder al sistema suplantando nuestra identidad.

```
Ubuntu 24.04.5 LTS so-lab01 tty3

so-lab01 login: alumno
Password:
```

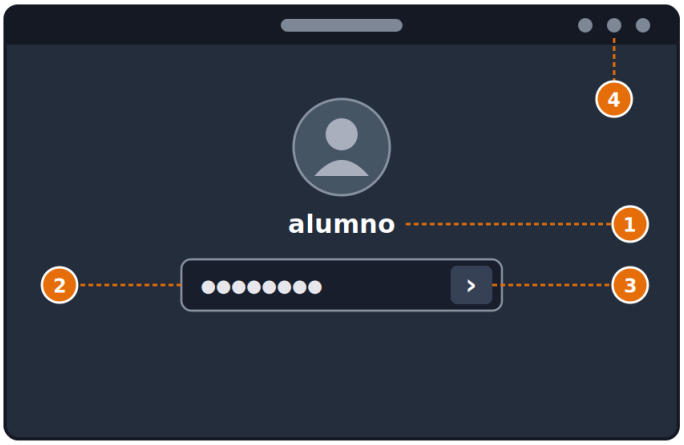
Es muy importante darse cuenta de que **UNIX distingue entre mayúsculas y minúsculas** (es *case sensitive*), por lo que hay que tener cuidado al teclear la clave. Si la clave es `Woody.Al`, no se admitirá como clave `woody.al` o `WOODY.AL`. Si tecleamos correctamente las credenciales de acceso, habremos iniciado una sesión con la máquina y se nos mostrará un mensaje de bienvenida y un indicador (***prompt***) que indica que el intérprete de órdenes (***shell***) está listo para que introduzcamos alguna orden.

El indicador que se muestra de petición de orden depende del intérprete de órdenes concreto que se esté usando, siendo indicadores típicos el signo de dólar (**$**, para la shell llamada `bash` - *Bourne Again Shell*) o el signo de porcentaje (**%**, para la shell `tcsh` - *Tenex style C Shell*).

```
so-lab01 login: alumno
Password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-… x86_64)
[…]
alumno@so-lab01:~$ _
```

Cuando un administrador crea una cuenta de usuario asigna una clave de acceso por defecto. El usuario puede cambiarla cuando lo desee ejecutando la orden `passwd` en el intérprete de órdenes. Esta orden preguntará la clave de acceso actual y luego solicitará dos veces la clave nueva, la segunda de ellas para confirmar que está bien escrita. Si el sistema operativo ha arrancado en modo interfaz gráfica de usuario (en systemd, el objetivo graphical.target)[2], el proceso de solicitar las credenciales se realiza en una ventana gráfica.

[2] **systemd y graphical.target:** `systemd` es el administrador del sistema en Linux que gestiona el arranque de servicios. El `graphical.target` (equivalente al antiguo *Runlevel 5*) arranca el sistema en **modo gráfico multiusuario con red activa e interfaz visual de escritorio**, cargando todos los servicios de `multi-user.target` más el servidor de pantalla.



1 **Nombre de usuario**
El identificador con el que el sistema le reconoce (login).

2 **Contraseña**
Cada carácter se sustituye por un punto al teclearlo.

3 **Confirmar**
Botón de acceso; equivale a pulsar la tecla Intro.

4 **Menú del sistema**
Accesibilidad, teclado y apagado del equipo.

Esquema: la disposición y el aspecto reales varían según la versión de Ubuntu.

Para saber más sobre el proceso de arranque en Ubuntu Linux consulte el Proceso de Inicio en las páginas de ayuda de Ubuntu.
En la bibliografía clásica encontrará que estos dos modos se denominan niveles de ejecución (runlevels) y se numeran del 0 al 6, nomenclatura heredada de System V. Ubuntu utiliza systemd como gestor de servicios, que habla de objetivos (targets). Puede consultar cuál se utiliza al arrancar su equipo con la orden:

```
$ systemctl get-default
graphical.target
```

Desde la interfaz gráfica puede pasar a una consola de texto con la combinación Ctrl+Alt+F3 y volver a la sesión gráfica con Ctrl+Alt+F2 (el número de la consola gráfica puede variar según el equipo).

2. Salida del sistema

Si quiere terminar la sesión y salir del sistema, pruebe a teclear `exit`, `logout`, o pulse `ctrl-d`. Alguno de estos métodos funcionará en su sistema. La salida vía `ctrl-d` muchas veces se desactiva para evitar desconexiones accidentales.
Si se ha arrancado Ubuntu Linux en modo de interfaz gráfica interactiva, la desconexión del sistema se realiza desde el menú del sistema, en la esquina superior derecha: el botón de apagado despliega, entre otras, la opción «Cerrar la sesión».



1. **Menú del sistema**
   Se abre al pulsar los iconos de la esquina superior derecha.
2. **Configuración**
   Abre la aplicación de ajustes del sistema.
3. **Bloquear**
   Bloquea la pantalla sin cerrar la sesión.
4. **Apagado**
   Despliega las opciones de apagado y de sesión.
5. **Cerrar la sesión**
   Termina la sesión sin apagar el equipo.

Esquema: la disposición y el aspecto reales varían según la versión de Ubuntu.

3. Introducción de órdenes

Una vez que el sistema muestra el indicador, se pueden introducir **órdenes** o **mandatos** (*commands*).

La sintaxis general o estructura de una orden es la siguiente:

$ orden [[-]opcion(es)] [argumento(s) de opcion(es)] [argumento(s) de orden(es)] donde:

- **$** es el indicador de petición de orden de la shell,
- cualquier cosa encerrada entre **[]** es opcional,
- **orden** es el nombre de una orden LINUX válida para ese shell, escrita con letras minúsculas,
- **[[-]opcion(es)]** es uno o más modificadores que cambian el comportamiento de la orden,
- **[argumento(s) de opcion(es)]** es uno o más modificadores que cambian el comportamiento de las opciones, y
- **[argumento(s) de orden(es)]** es uno o más objetos, separados por espacios, que serán afectados por la orden.

Para iniciar un terminal en el caso de que Ubuntu Linux haya arrancado en modo interfaz gráfica interactiva, se abre la vista de Actividades, con el botón de la esquina superior izquierda o con la tecla Super. En el espacio para buscar se escriben las primeras letras de la palabra ***Terminal*** (en este caso no hace falta distinguir entre mayúsculas y minúsculas) y el buscador mostrará la aplicación Terminal, que es la forma de invocar al intérprete de órdenes en Ubuntu Linux. También se puede utilizar la combinación de teclas `ctrl+alt+t`.



1 **Actividades**
Botón de la esquina superior izquierda, o tecla Super.

2 **Buscador**
Basta con escribir las primeras letras del nombre.

3 **Terminal**
Al pulsarlo se abre el intérprete de órdenes.

4 **Mostrar aplicaciones**
Lista completa, desde el dock de Ubuntu.

Atajo: Ctrl+Alt+T
Abre el terminal directamente.

Esquema: la disposición y el aspecto reales varían según la versión de Ubuntu.

En Ubuntu 26.04 LTS. El terminal predeterminado es Ptyxis, que sustituye a GNOME Terminal. Se encuentra igualmente buscando «terminal» y se abre con la misma combinación de teclas; su aspecto es distinto al de las imágenes de este guión.

Para introducir una orden se escribe el nombre de la orden seguida de cero o más argumentos separados por espacios en blanco y/o tabuladores, y seguido de la pulsación de la tecla *Enter*.

Por ejemplo, la orden **ls** (abreviatura de *list*) sirve para listar los ficheros y directorios que cuelgan de los directorios. Si la utilizamos sin argumentos lista o enumera los ficheros y directorios que cuelgan del **directorio de trabajo** (**el directorio en el que nos encontramos situados**) ordenados lexicográficamente.

```
alumno@so-lab01:~/Documentos$ ls
archivo.txt  otros_programas  proceso  proceso.c
```

Al escribir en un terminal, **ls** colorea la salida: los directorios en azul y los ficheros ejecutables en verde. El color lo añade la opción `--color=auto`, incluida en el alias que Ubuntu define para ls; puede comprobarlo con la orden alias ls y observar que el color desaparece al redirigir la salida a un fichero.

Pasemos a comentar algo sobre los **argumentos** o **parámetros de las órdenes**. Existen dos tipos de parámetros: **los fijos y los variables**.

- Los **parámetros fijos** siempre toman el mismo valor, indican el modo de funcionamiento que se desea de la orden. En la mayoría de los casos van después de la orden, precedidos por el símbolo del guión (-), por ejemplo:

La opción **l** produce un listado más completo (*long*), con una línea por ítem, en cada línea se especifican datos sobre cada ítem. Si se quiere utilizar varios parámetros fijos en una orden se puede poner un guión y poner todas las opciones seguidas, o preceder cada opción por un guión. Por ejemplo, las órdenes **ls -lF** y **ls -l -F** son equivalentes y producen el listado:

```
alumno@so-lab01:~/Documentos$ ls -l
total 28
-rw-rw-r-- 1 alumno alumno    15 sep 14 09:40 archivo.txt
drwxrwxr-x 2 alumno alumno  4096 sep 16 14:21 otros_programas
-rwxrwxr-x 1 alumno alumno 15776 sep 10 13:08 proceso
-rw-rw-r-- 1 alumno alumno    45 sep 10 13:08 proceso.c
```

La opción **F** coloca una diagonal '/' después de cada fichero que es un directorio, y un asterisco '*' después de cada fichero ejecutable.

- Por último, están los **parámetros variables**, como su nombre indica su valor no es fijo.

Así, la orden **ls** admite como parámetro un nombre de directorio, **ls** listará todos los ficheros que cuelguen de dicho directorio. Por ejemplo, **ls -l /bin** producirá un listado largo de todos los ficheros que cuelgan del directorio */bin*.

```
alumno@so-lab01:~/Documentos$ ls -lF
total 28
-rw-rw-r-- 1 alumno alumno    15 sep 14 09:40 archivo.txt
drwxrwxr-x 2 alumno alumno  4096 sep 16 14:21 otros_programas/
-rwxrwxr-x 1 alumno alumno 15776 sep 10 13:08 proceso*
-rw-rw-r-- 1 alumno alumno    45 sep 10 13:08 proceso.c
```

La opción **a** (*all*) hace que aparezcan en el listado los **archivos y directorios ocultos** (su nombre comienza por un punto **.**). Los archivos y directorios ocultos suelen ser de configuración.

```
alumno@so-lab01:~/Documentos$ ls -al
total 40
drwxrwxr-x  3 alumno alumno  4096 sep 16 14:27 .
drwxr-x--- 17 alumno alumno  4096 sep 21 09:45 ..
-rw-rw-r--  1 alumno alumno    15 sep 14 09:40 archivo.txt
-rw-rw-r--  1 alumno alumno     5 sep 16 14:27 .oculto
drwxrwxr-x  2 alumno alumno  4096 sep 16 14:21 otros_programas
-rwxrwxr-x  1 alumno alumno 15776 sep 10 13:08 proceso
-rw-rw-r--  1 alumno alumno    45 sep 10 13:08 proceso.c
```

Uno de los defectos de LINUX es la **falta de consistencia en los argumentos de las órdenes**.

- Así, en la mayoría de las órdenes los **argumentos fijos pueden ponerse antes o después de los variables**, pero hay órdenes en las que esto no ocurre.
- Otro inconveniente aparece cuando se utilizan varios argumentos fijos en una orden; **algunas órdenes permiten especificarlos todos tras un guión o utilizar un guión por cada argumento fijo; otras órdenes, sin embargo, sólo permiten una de las opciones**.
- Por último, **algunas letras utilizadas para especificar argumentos fijos representan funciones distintas en órdenes distintas**; por ejemplo, **-r** significa la

utilización recursiva de ciertas órdenes, mientras que otras, que no incluyen la opción –r, su utilización recursiva se especifica mediante **-s**.

4. Algunas órdenes útiles

En este apartado se van a comentar algunas de las órdenes que se utilizan con mucha frecuencia, de este modo el lector curioso podrá inspeccionar los ficheros del sistema antes de que estudiemos con más rigor estas órdenes.

En Ubuntu 26.04 LTS. Las utilidades básicas del sistema (ls, head, sort, wc, cut…) ya no son las de GNU, sino una reimplementación del proyecto uutils. El comportamiento pretende ser idéntico, pero puede haber diferencias en los mensajes y en alguna opción poco habitual. Compruebe cuál está usando con ls --version. Si alguna orden de este guión no se comporta como aquí se describe, repítala con el prefijo gnu (por ejemplo, gnuls en lugar de ls) y anote la diferencia.

- **pwd**: muestra el directorio de trabajo (*Print Working Directory*).
- **cd [*dir*]**: sirve para cambiar el directorio (*Change Directory*) de trabajo. Si se utiliza sin argumentos cambia el **directorio de trabajo** al **directorio base** (*home directory*). Hay que tener en cuenta que en UNIX se usa la diagonal ('/'), en lugar de la diagonal invertida ('\') utilizada en Windows, en las trayectorias, rutas o caminos (*paths*) de los ficheros. La ruta \ab\c de Windows se escribe /ab/c en UNIX.
- **more *fich***: el programa **more** es un paginador, es decir, un programa que muestra las páginas de un fichero de texto. Los paginadores no están normalizados, por tanto, es posible que en su sistema no tenga el programa **more** (en Ubuntu dispone siempre de **less**, más completo, que es además el paginador que utiliza man). Si el texto del fichero no cabe en la pantalla, pulsando *Enter* se puede ver la siguiente línea del fichero, y pulsando el espaciador una pantalla más del fichero. Una vez que ha ejecutado more puede obtener una ayuda si pulsa la letra **h**.
- **cat fich**: concatena uno o más ficheros y muestra su contenido en la salida.
- **cp**: sirve para copiar (*copy*) ficheros.
- **mv**: sirve para desplazar (*move*) un fichero en el sistema de ficheros.
- **rm**: sirve para borrar -*remove*- ficheros (para ser exactos borra enlaces como veremos más adelante).
- **man *orden***: UNIX tiene un manual para consultarlo en línea desde el terminal del ordenador. Teclee **man cp** y obtendrá información sobre la orden **cp**. Escriba **man man** y obtendrá ayuda sobre el manual. El manual también incluye información sobre llamadas al sistema, rutinas de biblioteca de C, juegos, y más cosas. El manual está dividido en secciones numeradas: la 1 contiene las órdenes, la 2 las llamadas al sistema y la 3 las funciones de biblioteca. Cuando un mismo nombre existe en varias secciones, se indica cuál se quiere consultar: man 2 fork, man 3 printf. La orden **apropos** palabra busca palabra en las descripciones de todas las páginas.
- **touch:** Este comando sirve para crear un nuevo archivo vacío si este no existe.

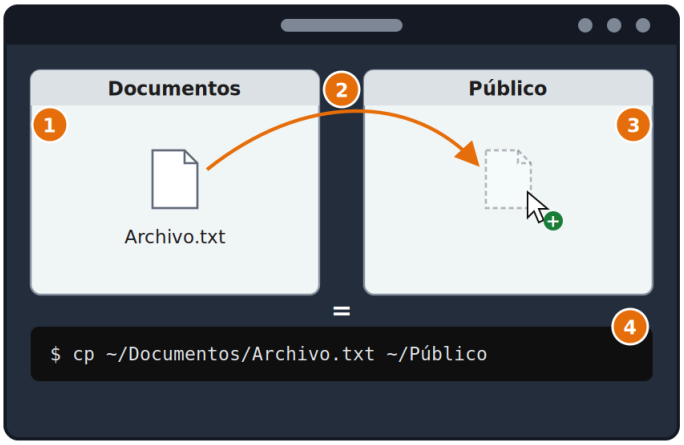
- **whatis *orden/es***: con esta orden se puede obtener una descripción abreviada de lo que hace cualquier orden. Se pueden pasar más de un argumento en la misma línea de órdenes separadas por espacios para consultar varias órdenes.
- **whereis orden/es:** se usa para localizar los archivos binarios, de código fuente y de manual de un comando específico en tu sistema.
- **whoami**: muestra en pantalla nuestra identidad de usuario.
- **hostname**: muestra el nombre de la computadora al que nos hemos conectado.
- **uname**: muestra información relacionada con el sistema operativo que se ejecuta en el computador.
- **wc**: (*word count*) lee la entrada estándar y cuenta bytes (wc -c), caracteres (wc -m), palabras (**wc -w**) o líneas (**wc -l)**.
- **echo**: mostrar una línea de texto/cadena en una salida estándar. Con la opción **-n** se evita saltar de línea. Con la opción **-e** se pueden incluir los caracteres con la barra invertida para saltar de línea **\n** o incluir un tabulador **\t** o retroceder un carácter con \b. Las opciones de **echo** no son iguales en todos los intérpretes; cuando se necesite un resultado portable conviene utilizar printf.
    - ```
      echo "Printing text with newline"
      echo -n "Printing text without newline"
      echo -e "\nRemoving \t backslash \t characters\n"
      ```

5. El Shell

Cuando el sistema ha arrancado en modo consola y se efectúa la conexión, el sistema LINUX pone en marcha **un programa que actúa como interfaz entre nosotros y el sistema operativo**. En consecuencia, cuando el sistema despliega el indicador '$' y se teclean órdenes que son ejecutadas por el sistema, no es el sistema operativo el que se está dirigiendo al usuario, sino un intermediario llamado **intérprete de órdenes** o ***shell***.

Intérprete de órdenes, Intérprete de mandatos, Intérprete de línea de mandatos, Intérprete de comandos, Terminal, Consola, Shell o su acrónimo en idioma inglés **CLI** por ***Command line interface***, es un programa informático que actúa como Interfaz de usuario para comunicar al usuario con el sistema operativo mediante una ventana que espera órdenes escritas por el usuario en el teclado (por ej. la orden **`sort -n listatelefonos > listatelefonos.ordenada`**), los interpreta y los entrega al sistema operativo para su ejecución. La respuesta del sistema operativo es mostrada al usuario en la misma ventana. A continuación, el programa *shell* queda esperando más instrucciones.

Cuando el sistema ha arrancado en modo interfaz gráfica interactiva, la propia interfaz permite comunicación entre usuario y sistema operativo. Así, si un usuario arrastra (con la tecla `Ctrl` pulsada) el icono de un documento de texto llamado `Archivo.txt` desde la carpeta `~/Documentos` a la carpeta `~/Público`, en realidad está llamado a la orden: **`cp ~/Documentos/Archivo.txt ~/Público`** La interfaz gráfica interpreta las acciones del usuario con el ratón sobre los iconos del sistema y la traduce a una orden para el sistema.
      ```



1 Carpeta de origen
~/Documentos, con el fichero que se quiere copiar.

2 Arrastrar con Ctrl
Con la tecla Ctrl pulsada, el arrastre copia en vez de mover.

3 Carpeta de destino
~/Público, donde aparece la copia.

4 Orden equivalente
La interfaz gráfica traduce la acción a una orden del sistema.

Esquema: la disposición y el aspecto reales varían según la versión de Ubuntu.

5.1. El shell como ejecutor de órdenes

Cuando el shell despliega el indicador, el usuario teclea una orden (un programa), seguida de sus argumentos. Una de las misiones del shell es realizar las llamadas al sistema oportunas para ejecutar la orden tecleada, una de ellas lo convierte en proceso. Para ilustrar esto, a continuación se muestra el código de un programa que representa un shell muy simplificado (sólo admite órdenes sin modificadores ni argumentos de entrada):

Código en C++ de un shell simplificado disponible en https://github.com/SSOOUJA/practicas

```cpp
#include <iostream>
#include <string>       // Clase std::string y función std::getline()
#include <sys/wait.h>   // Incluye waitpid() para sincronización entre procesos
#include <sys/types.h> // Definiciones de tipos como pid_t
#include <unistd.h>     // Módulo para llamadas al sistema como fork() y execlp()

int main() {

   std::string comando; // variable para línea de comando introducida por usuario
   pid_t pid=0;          // variable para almacenar el ID del proceso hijo
   int status=0;         // almacena el estado de finalización del proceso hijo

   std::cout << "> "; // imprime el indicador >

   // Bucle lectura órdenes hasta el fin del archivo (Ctrl+D o un error de lectura)
   while ( std::getline(std::cin , comando) ) {

       pid = fork(); // Crea un nuevo proceso hijo

       if (pid < 0) { // Si fork() devuelve valor negativo, error al crear el hijo

           std::cerr << "Error en fork" << std::endl;
           return 127; // Termina con un código de error usando exit(127)

       } else if (pid == 0) { // Si pid es 0, se está ejecutando el proceso hijo

           /* El hijo intenta ejecutar el comando ingresado
             execlp busca el ejecutable en el path del sistema.
             Requiere cadenas en C (vectores de char terminados en valor 0 */
           execlp( comando.c_str() , comando.c_str() , (char *)nullptr);

           // Si execlp falla, muestra error
           std::cerr << "No se pudo ejecutar " << comando << std::endl;
           return 126; // Termina con un código de error usando exit(126)

       } else { // Si pid > 0, estamos ejecutando el proceso padre
```

```
38              // El padre espera que el hijo termine con waitpid()
39              if ((pid = waitpid(pid, &status, 0)) < 0) {
40                  // Si waitpid falla, muestra error
41                  std::cerr << "Error en waitpid" << std::endl;
42                  return 125;// Termina con un código de error usando exit(125)
43              }
44              std::cout << "> "; // Vuelve a mostrar el prompt solicitar otro comando
45          }
46      }
47      return 0; // Termina con un código de éxito usando exit(0)
48  }
```

Para poder ejecutar ese shell simplificado, en primer lugar hay que escribirlo o copiarlo a un fichero con nombre `shell.cpp` (es importante que la extensión sea `.cpp`, es un programa en lenguaje C++). A continuación hay que compilarlo y enlazarlo, eso se realiza mediante la orden: `g++ -std=c++17 -Wall -Wextra shell.cpp -o shell`. Por último hay que ejecutarlo mediante la orden `./shell`.

Aunque el comando simplificado `g++ shell.cpp -o shell` es técnicamente suficiente para generar el ejecutable, omitir los parámetros adicionales es arriesgado. Sin `-std=c++17`, el compilador utilizará una versión antigua de C++ y rechazará las funciones modernas del lenguaje. Además, prescindir de `-Wall -Wextra` oculta los avisos del compilador, permitiendo que errores lógicos graves (como variables sin inicializar o fallos de memoria) pasen desapercibidos y provoquen que el programa falle de forma impredecible durante su ejecución.

En la siguiente imagen puede verse cómo compilar y ejecutar el programa en un Terminal de Ubuntu Linux (en la última línea se ha pulsado **`Ctrl-d`**):

Si no tienes instalado el compilador de C++ **`g++`** que forma parte de la colección de compiladores GNU **`GCC`**, lo puedes instalar con el siguiente metapaquete, que instala tanto las herramientas de GCC como otras utilidades necesarias en el proceso de compilación, por ejemplo, make:
`sudo apt install build-essential`

Tal vez al intentar ejecutar el programa te has encontrado con el problema "**orden no encontrada**" o "**command not found**"

> **Nota:** ¿Por qué al ejecutar una orden en el shell me dice "orden no encontrada" o "command not found"? Consulta las Preguntas Frecuentes

```
alumno@so-lab01:~/SSOO$ g++ -std=c++17 -Wall -Wextra shell.cpp -o shell
alumno@so-lab01:~/SSOO$ ./shell
> ls
shell  shell.cpp
> whoami
alumno
> pwd
/home/alumno/SSOO
> minecraft
No se pudo ejecutar minecraft
> alumno@so-lab01:~/SSOO$
```

A continuación se comentan ciertos aspectos del programa:

- [Línea 16] Se utiliza la función de E/S estándar `getline` para leer una línea de una vez de la entrada estándar hasta que presionas la tecla Enter. Cuando se escribe el carácter fin de fichero (o carácter **EOF**, del inglés **E**nd **O**f **F**ile), el cual normalmente es `ctrl-d`, como el primer carácter de una línea, `std::getline devuelve el propio flujo (std::cin), que al evaluarse como condición del while es falso, el ciclo se detiene`, y el proceso finaliza.
- [Línea 18] Se llama a **`fork`** para **crear un proceso nuevo**. El nuevo proceso es una copia del proceso llamador (su código, su datos, su pila y su Bloque de Control de Proceso que tendrá otro identificador, otro PID), y se dice que el proceso que llama es el padre y el proceso recién creado es el hijo. **`fork`** devuelve el identificador de proceso (un entero no negativo) del nuevo proceso hijo al padre, y devuelve 0 al hijo. Puesto que `fork` crea un nuevo proceso, se dice que es llamado una vez (por el padre) y que devuelve un valor dos veces (tanto en el padre como en el hijo).
  Cuando se hace la llamada a `fork`, se copia el proceso en el ESTADO en el que se encuentran. Por tanto, tanto padre como hijo retoman la ejecución (cuando el SO le asigne la CPU, eso ocurrirá cuando decida el planificador) en la asignación a la variable `pid`. La variable `pid` ocupará dos casillas de memoria distintas pertenecientes al espacio de memoria en un caso del proceso padre y en el otro del proceso hijo.
- [Líneas 23, 34, 41 y 46] los procesos finalizan devolviendo un código de estado con el que indican si han podido realizar la tarea asignada. Por convención se utiliza 0 para indicar que la operación se ha realizado correctamente y valores diferentes a 0 en caso contrario. Los procesos indican su finalización al SSOO con la función **`exit`** pasándole a la misma el código de estado que quieren notificar. No obstante, en C/C++ es conveniente finalizar la función main con *`return`* para garantizar que el programa pueda liberar adecuadamente los recursos gestionados, e.g. cerrar archivos, destruir objetos, etc. Una vez realizadas estas tareas, el programa llamará automáticamente a **`exit()`** pasándole el código devuelto con **`return`**.
- [Línea 30] En el hijo se llama a **`execlp`** para ejecutar la orden que fue leída de la entrada estándar. Esto reemplaza el código del proceso hijo por el fichero del nuevo programa. La combinación de un **`fork`**, seguido de un *`exec`* (*`execlp`* es un caso particular de la llamada al sistema *exec*) es lo que algunos sistemas operativos llaman intercambio de un nuevo proceso. En UNIX las dos partes se separan en funciones individuales.

Es muy importante darse cuenta de que **`execlp` reemplaza** al proceso que la llama (el proceso hijo), por tanto sólo si falla se ejecutan las líneas 33 y 34. En caso contrario, esas líneas nunca se ejecutan.

- [Línea 38] Ya que el hijo llama a **`execlp`** para ejecutar el fichero del nuevo programa, el padre necesita esperar a que termine el hijo. Esto se hace llamando a **`waitpid`**, especificando a qué proceso deseamos esperar (el argumento ***pid***, que es el identificador de proceso del hijo). La función **`waitpid`** también devuelve el estado de terminación del hijo (en la variable ***status***), pero en este *shell* tan simple no se hace nada con su valor. Se podría examinar para determinar cómo terminó el hijo.
- [Líneas 20 a 44] La llamada a la función **`fork()`** hace una copia del proceso que la llama. Por tanto se tienen dos procesos cuyas instrucciones de programa son idénticas. Si se quiere que realicen tareas distintas, la única solución es introducir instrucciones de programa que averigüen de qué proceso se trata, y en función de si es el padre o el hijo, que realicen una determinada tarea u otra.

  Esta es la misión del `if - then - else`.

  La primera parte del `if` pregunta si la variable `pid` es menor que cero. Si eso pasa es porque la llamada a **`fork()`** no ha podido crear el subproceso.

  La segunda parte del `if` pregunta si la variable `pid` es cero. Si lo es, es porque esa instrucción la está ejecutando el subproceso o proceso hijo. Recuerda que **`fork()`** devuelve cero al subproceso.

  La tercera parte del if no necesita preguntar nada, ya que **`pid`** sólo puede ser mayor que cero. Esto indica que esa instrucción la está ejecutando el proceso padre. **`fork()`** devuelve al proceso padre el id del subproceso que crea.
- La limitación fundamental de este programa es la de que el usuario no puede pasar argumentos a la orden que desea ejecutar. No se puede, por ejemplo, especificar el nombre de un directorio a listar. Solamente se puede ejecutar **`ls`** sin argumentos (por lo tanto, **`ls`** actuará sobre el directorio de trabajo). Para permitir argumentos se requeriría analizar sintácticamente la línea de entrada, separándolos, y pasándole cada argumento de la orden como un argumento separado a la función *execlp*.

Comprender cómo funciona un shell por dentro no es fácil sobre todo si no se entiende qué es un proceso.

Nota: Puede que sigas preguntándote ¿para qué sirven las llamadas fork y wait? Consulta las preguntas frecuentes.

5.2. El shell como intérprete de metacaracteres

El shell que hemos visto en el apartado anterior era muy simple, una primera mejora ya se apuntó más arriba, consiste en **analizar la línea de entrada para pasarle los argumentos a las órdenes.**
Sin embargo, antes de pasar los argumentos a las órdenes el propio shell realiza una interpretación de ciertos caracteres de la línea de entrada. Tras realizar dicha interpretación procede a pasarle los argumentos (posiblemente distintos a los originales debido a la interpretación) a la orden. Los caracteres que el shell interpreta se llaman **metacaracteres**.

A continuación vamos a estudiar dos tipos de metacaracteres que interpretan todos los shells importantes, en concreto son:

- Los **metacaracteres mágicos o comodines**. Sirven para especificar un conjunto de ficheros indicando solamente el patrón de sus nombres. El shell encontrará los ficheros que verifiquen el patrón.
- Los **metacaracteres de redireccionamiento de la E/S**. Se puede lograr que la salida de cualquier programa se dirija hacia un fichero, en lugar de dirigirse hacia el terminal; se puede hacer que la entrada provenga también de un fichero y no del terminal. La entrada y la salida incluso pueden conectarse a otros programas.

5.3. Abreviaturas de nombres de ficheros

Comencemos con los patrones de nombres de ficheros. Suponga que se está tecleando un documento grande, por ejemplo un libro. Éste se divide en partes de menor tamaño, como capítulos y, tal vez, secciones. También físicamente habrá de estar dividido, debido a la incomodidad que representa la edición de ficheros extensos. Así, el documento debe ser tecleado en varios ficheros. Se podrían tener ficheros separados para cada capítulo, llamados cap1, cap2, etc. Asimismo, si cada capítulo fuera dividido en secciones, podrían crearse los siguientes ficheros:

```
cap1.1

cap1.2

cap1.3

. . .

cap2.1

cap2.2
```

Es conveniente que ejecute el siguiente programa (se lo puede bajar a su directorio de trabajo desde el siguiente repositorio en github https://github.com/SSOOUJA/practicas/ o cambiadirectamente desde los siguientes enlaces[3]:

Fichero/programa en bash

- fichero creacap
- fichero createmp

Descarga el fichero **creacap** y cambia sus permisos con **chmod +x creacap** para hacerlo ejecutable y ejecútalo.

```
$ ./creacap
```

y se crearán en su directorio 30 ficheros de nombre **capi, capi.1 y capi.2** para **i** desde 1 hasta 10. El contenido de los ficheros capx y capx.y es la cadena de caracteres capítulox y capx.y respectivamente.

Con una nomenclatura sistemática se puede saber rápidamente dónde encaja un fichero dentro del total. ¿Y si el usuario quisiera ver el contenido de todo el libro? Podría teclear

```
$ cat cap1.1 cap1.2 cap1.3...
```

ya que la orden cat tiene como efecto mostrar en la pantalla el contenido de los ficheros que se le pasen como argumentos. Sin embargo, el usuario pronto se aburriría de teclear tantos nombres de fichero y empezaría a cometer errores. Aquí es donde entran en acción las abreviaturas de nombres de fichero. Si se teclea

```
$ cat cap*
```

el **shell** toma el * como "cualquier cadena de caracteres", de modo que **cap*** es un patrón que corresponde a todos los nombres de fichero en el directorio de trabajo que empiecen con cap. El shell crea la lista, ordenada según el criterio de la configuración regional de la sesión (variable LC_COLLATE), que no coincide necesariamente con el orden del código ASCII y la pasa a la orden **cat**. La orden **cat** nunca ve el ***.** La búsqueda que hace el shell en el directorio de trabajo genera una lista de cadenas que se pasan a **cat**.

El punto crucial es que la abreviatura del nombre de fichero no es propiedad de la orden **cat**, sino un servicio, un metacaracter o comodín del shell. Por ello, puede utilizarse para generar una secuencia de nombres de ficheros destinados a cualquier orden. Existe un programa u orden llamado **echo** que es especialmente útil para ensayar el significado de los comodines. Como puede suponerse, **echo** no hace otra cosa que retransmitir ("hacer eco a") sus argumentos al terminal:

[3] Al ser ficheros de texto, el navegador por defecto mostrará su contenido. Para guardar el fichero en disco pulsa con el botón derecho sobre su contenido en el navegador y selecciona "Guardar como". Otra opción más "avanzada" consiste en copiar el enlace del fichero y utilizar la utilidad *wget* para descargarlo desde una shell en linux, ejemplo: `$ wget https://github.com/SSOOUJA...`

```
$ echo hello world
hello world
```

Pero los argumentos pueden ser generados por medio de patrones:

```
$ echo cap1.*
```

lista los nombres de todos los ficheros del capítulo 1,

```
$ echo *
```

lista por orden alfabético todos los nombres de ficheros que se encuentren en el directorio de trabajo,

```
$ cat *
```

muestra en la pantalla el contenido de todos los ficheros del usuario (en orden lexicográfico), y

```
$ rm *
```

borra todos los ficheros del directorio de trabajo (se debe estar muy seguro de que esto es lo que se desea hacer). El ***** no está limitado a la última posición en un nombre de fichero. Se pueden colocar uno o varios de ellos en cualquier posición del nombre de fichero. Así,

```
$ rm *.save
```

elimina todos los ficheros que terminan en **.save**.

Observe que los nombres de fichero se ordenan lexicográficamente, lo cual no es lo mismo que si estuvieran ordenados numéricamente. Si el libro tiene 10 capítulos, el orden tal vez no sea el deseado por el usuario, ya que cap10 queda antes que cap2:

```
$ echo *
cap1 cap10 cap10.1 cap10.2 cap1.1 cap1.2 cap2 cap2.1 cap2.2 ...
```

El criterio de ordenación depende de la configuración regional. Con la configuración española, la salida anterior no sigue el orden del código ASCII: los signos de puntuación se ignoran en una primera comparación, de modo que cap1.1 se ordena como si fuera cap11, después de cap10; con LC_ALL=C, cap1.1 aparece antes que cap10. La diferencia se aprecia también al ordenar palabras que empiezan por mayúscula y por minúscula con la configuración de su sesión y con la configuración C:

```
$ printf 'Banana\napple\nCereza\n' | sort
apple
Banana
Cereza
$ printf 'Banana\napple\nCereza\n' | LC_ALL=C sort
Banana
Cereza
apple
```

El ***** no es la única característica de reconocimiento de patrones del shell, aunque sea la utilizada más frecuentemente. El patrón [...] representa cualquiera de los caracteres dentro de los corchetes. Podemos abreviar un rango de letras o dígitos consecutivos:

```
$ cat cap[12346789]*  # Mostrar los capítulos 1, 2, 3, 4, 6, 7, 8, y 9, pero no
el 5
$ cat cap[1-46-9]*  # Lo mismo que antes
$ rm temp[a-z]  # Elimina los tempa, ..., tempz que existan
```

El patrón **?** representa a cualquier carácter:

```
$ ls ?  # Mostrar los ficheros con nombre de un sólo carácter
$ ls -l cap?.1  # Mostrar cap1.1 cap2.1 cap3.1 etc, pero no cap10.1
$ rm temp?  # Eliminar los ficheros temp1, ..., tempa, etc.
```

Nótese que estos patrones funcionan sólo con nombres de ficheros existentes (pues el shell iguala el patrón con ficheros que existen). En particular, no es posible crear nuevos ficheros usando patrones. Por ejemplo, si se desea expandir `cap` a `capitulo` en cada nombre de fichero no puede hacerse de la siguiente manera:

```
$ mv cap* capitulo*  # ¡No funciona!
```

ya que **`capitulo*`** no concuerda con ningún fichero existente. El comportamiento de los patrones o comodines (*wildcards*) como ***** y **?** en el shell de Linux es que expanden el patrón a una lista de nombres de ficheros que ya existen en el directorio actual. Esto sí se puede hacer en MS DOS con la orden move ya que el intérprete de órdenes de MS DOS (el command.com) no interpreta el *, sino que le pasa a la orden move como argumentos cap* y capítulo*, siendo move la encargada de interpretar el asterisco.

Los caracteres de patrones tales como ***** pueden usarse tanto en trayectorias/rutas como en nombres de ficheros simples; la búsqueda se hace para cada componente de la trayectoria que contenga un metacaracter *****. Así, **/usr/mary/*** lleva a cabo la búsqueda dentro de **`/usr/mary`**, y **`/usr/*/calendar`** genera una lista de nombres de todos los ficheros **`calendar`** que cuelgan de los subdirectorios de **`/usr`**.

Si alguna vez desea eliminar el significado especial de `*`, `?`, etc., deberá encerrarse el argumento completo entre apóstrofos, como en

```
$ ls '?'
```

esto también puede hacerse anteponiendo al metacarácter una diagonal invertida:

```
$ ls \?
```

ambas órdenes mostrarán un fichero cuyo nombre fuera ?, si existe uno con tal nombre.

Ejemplos usando los comodines * ?

```
$ cd */logs  # Entra en el directorio logs de cualquier subdirectorio; si hay varios, cd falla
$ cat *.txt  # Concatena y muestra el contenido de todos los archivos con extensión .txt
$ cat *log | grep error  # Muestra las líneas que contienen "error" en todos los archivos .log
$ apropos grep  # Busca «grep» en las descripciones de todas las páginas del manual
$ whereis vim  # Localiza el ejecutable, las fuentes y la página de manual de vim
$ touch archivo_{1..10}.txt  # Crea 10 archivos llamados archivo_1.txt, archivo_2.txt, etc.
$ wc -l *.cpp  # Cuenta las líneas de todos los archivos con extensión .cpp
```

5.4. Redireccionamiento de la E/S

En el sistema operativo UNIX toda la entrada/salida se realiza leyendo o escribiendo en ficheros, ya que **todos los periféricos, incluso los terminales, son ficheros del sistema de ficheros**. Esto significa que una interfaz sencilla y homogénea gestiona toda la comunicación entre un programa y los periféricos.

Cuando empieza la ejecución de un programa en UNIX éste se inicia con tres ficheros abiertos:

- **la entrada estándar (`stdin`)** File Descriptor 0 ,
- **la salida estándar (`stdout`)** File Descriptor 1 y
- **la salida de error estándar (`stderr`)** File Descriptor 2 (algunas veces llamada error estándar).

Estos tres ficheros están asociados por defecto al terminal, por lo que **la lectura de la entrada estándar equivale a leer del teclado y la escritura en la salida y el error estándar equivale a escribir en la pantalla**.

El lenguaje C++ dispone del módulo *iostream* para la manipulación de E/S. En dicho módulo se pueden encontrar objetos de la jerarquía de **clases f*stream*** que disponen de métodos para interaccionar con la entrada estándar, **std::cin**, salida estándar, **std::cout** y salida de error estándar **std::cerr** ya estudiados en las asignaturas de Fundamentos de Informática y Programación Orientada a Objetos.

Nota: se puede repasar el uso archivos en C++ en el tema 10 del libro de Fundamentos básicos de programación en C++

A continuación vamos a ver un ejemplo de un programa en C++ que utiliza la entrada/salida estándar; su nombre es **`mayus`**, su código fuente **`mayus.cpp`**, y traduce su entrada a mayúsculas:

```
#include <iostream> // Módulo de E/S y acceso a cin, cout y cerr
#include <cctype>   // islower() y toupper()

int main() {

   int c = 0;

   while ((c = std::cin.get()) != EOF)
   {
       std::cout.put( std::islower(c)? std::toupper(c) : c );
   }

   return 0;
}
```

Comentemos algunas de las funciones utilizadas:

- El método **`cin.get()`** devuelve el siguiente carácter de la entrada estándar cada vez que se le llama, y devuelve EOF cuando encuentra el fin de fichero en la entrada. La biblioteca estándar define la constante simbólica EOF como -1, pero las comparaciones deben realizarse en términos de EOF y no de -1 para hacerlas independientes del valor específico. Por ese motivo la variable c se declara de tipo int y no char: get() devuelve un int capaz de representar los 256 valores posibles de un byte y, además, el valor EOF. Si se guardara en un char, el byte 0xFF se confundiría con EOF y el programa dejaría de leer al encontrarlo.
- **cout.put()** envía a la salida estándar el carácter correspondiente al código ascii que se le pasa como parámetro
- La función **`islower`** comprueba si un argumento es una letra minúscula, devolviendo un código ascii distinto de cero en caso afirmativo, o cero en caso contrario. La función **`toupper`** convierte una letra minúscula en su mayúscula correspondiente.

Compilamos el programa y probemos su funcionamiento:

```
$ g++ -std=c++17 -Wall -Wextra -o mayus mayus.cpp
$ ./mayus
Cogito ergo sum
COGITO ERGO SUM
ctrl-d
```

El programa, efectivamente, transforma su entrada estándar a mayúsculas. La entrada, salida y error estándar (asignados por defecto al terminal) se pueden redireccionar a ficheros (y como hay ficheros que representan periféricos, también a periféricos) utilizando los metacaracteres de redireccionamiento. El carácter '**>**' seguido de una trayectoria de fichero sirve para redireccionar la salida estándar al fichero especificado tras '**>**'.

```
$ ./mayus >borrar
Je pense donc je suis
ctrl-d
$ cat borrar
JE PENSE DONC JE SUIS
```

La orden **cat** seguida de un nombre de fichero de texto muestra en la salida estándar (el terminal por defecto) el contenido del fichero. El shell interpreta el metacarácter '**>**', realizando varias llamadas al sistema para que la salida estándar cambie del terminal al fichero borrar. El cambio de la salida estándar al fichero **borrar** es ajeno al programa **mayus**; en concreto la cadena '**>borrar**' no se incluye en los argumentos **argv** de la línea de órdenes. Para redireccionar la entrada estándar se utiliza el metacarácter '**<**' seguido de una trayectoria de fichero, esto hace que la entrada estándar del programa proceda de dicho fichero en lugar del terminal. Por ejemplo, creemos un fichero llamado **borrar2** que contiene una única línea con el texto "**Pienso luego existo**" y tecleamos

```
$ ./mayus <borrar2
PIENSO LUEGO EXISTO
```

Al igual que antes el shell es el encargado del redireccionamiento; en este caso se ha redireccionado la entrada estándar para que provenga del fichero **borrar2** en lugar del terminal (como antes, el programa **mayus** no percibe que su entrada ha sido redireccionada). Se puede redireccionar tanto la entrada como la salida estándar de una misma orden, como en '**mayus <borrar2 >borrar3**'. Existen muchas órdenes que leen y/o escriben en/de los ficheros estándares. Por ejemplo **ls**

```
$ ls
```

genera una lista de nombres de ficheros en el terminal del usuario. Pero si se teclea

```
$ ls >listaficheros
```

la misma lista de nombres de ficheros será colocada en el fichero **listaficheros**. El metacaracter '**>**' provoca que el fichero se cree en caso de no existir y, si ya existía, su contenido será reemplazado (hay que estar seguro al utilizar '**>fich**' de que si existe fich no nos interese su contenido). El símbolo '**>>**' opera igual que '>', excepto que aquel significa "anexa o añade al final de". Esto es,

```
$ ./mayus <borrar2 >>borrar1
```

pasa el contenido de **borrar2** a mayúsculas al final de lo que se encuentre en el fichero **borrar1**, en vez de reemplazar el contenido existente. Como en '>', si **borrar1** no existe se creará inicialmente vacío para el usuario.

En todos estos ejemplos, los espacios en blanco son opcionales a ambos lados de <, > o >>, si bien nuestro formato es el tradicional. Estos metacaracteres de redireccionamiento suelen coincidir en todos los shells, aunque usted es libre de escribir un shell que utilice otros caracteres para el redireccionamiento. Para redirigir únicamente los mensajes de error

se utiliza el operador 2>, donde el 2 es el número de descriptor asociado al error estándar. Para enviar la salida y los errores al mismo fichero se emplea > fichero 2>&1, o la forma abreviada de bash &> fichero. En los intérpretes de la familia de csh esta última operación se escribía >& fichero, notación que todavía aparece en bibliografía antigua y que en bash no redirige sólo el error estándar.

Existen multitud de programas en UNIX, como **`mayus`**, que leen de su entrada estándar, la transforman y escriben la entrada transformada en la salida estándar. A estos programas se les llama **filtros**. Su implementación se asemeja al siguiente programa, que es una nueva versión de **`mayus`** llamada **`mayus2`**:

```
1   #include <iostream>
2   #include <fstream>
3   #include <cctype>
4
5   void mayus(std::istream &in);
6
7   /// @brief mayus2: convierte la entrada de minúsculas a mayúsculas */
8   int main(int argc, char *argv[]) {
9      if (argc == 1) /* ningún argumento, a mayúsculas la entrada estándar */
10     {
11         mayus(std::cin);
12     }
13     else
14     {
15        std::ifstream infile;
16        infile.open(argv[1]);
17
18       if ( ! infile.is_open() ) {
19         std::cerr << argv[0] << ": no puedo abrir " << argv[1] << std::endl;
20         return 1; // finalizar con código de error
21       }
22       else
23       {
24          mayus(infile);
25          infile.close();
26
27       }
28     }
29     // en C++ main finaliza por defecto con return 0;
30  }
31  /// @brief pasa a mayúsculas el fichero in a la salida estándar */
32  void mayus(std::istream &in) {
33     int c=0;
34     while ((c = in.get()) != EOF)
35     {
36         std::cout.put( std::islower(c)? std::toupper(c) : c );
       }
    }
```

Este programa merece algunos comentarios:

- Si el programa no recibe argumentos (**`argc == 1`**) transformará la entrada estándar a mayúsculas en la salida estándar. Si recibe un argumento, que debe ser un fichero existente, lo pasa a mayúsculas igualmente en la salida estándar. Si no entiendes argc y argv consulta Argumentos ¿Cómo se pasan argumentos a una orden? en Preguntas Frecuentes.

- Cuando se quiere utilizar un fichero en C++ se utiliza la clase **fstream**, aunque existen versiones especializadas como **ifstream**, para realizar solo operaciones de lectura, o **ofstream** para operaciones de escritura.
- Para utilizar un fichero es preciso realizar una operación de apertura para indicárselo al sistema operativo. Esto se realiza mediante el método **open** (que toma como argumento la trayectoria del fichero). En las sucesivas llamadas a métodos del objeto estas estarán vinculadas a la trayectoria del fichero. Cuando no se desee utilizar más el fichero habrá que cerrarlo con el método **close** para que el sistema operativo libere los buffers y demás información que debe mantener por cada fichero abierto. Como se dijo anteriormente existen tres ficheros abiertos al iniciar un programa: la entrada **cin** , salida **cout** y salida de error estándar **cerr**.
- Se utiliza el **método get** para leer el siguiente carácter de un fichero, esta función devuelve EOF cuando se alcanza el fin del fichero. **put** es la inversa de get y envía el carácter c al fichero.

A continuación le sugerimos que experimente con **`mayus2`**, pruebe por ejemplo:

```
$ ./mayus2
$ ./mayus2 <fich  # donde fich es el nombre de un fichero de texto
$ ./mayus2 fich
$ ./mayus2 no_existe  # no_existe nombre de un fichero que no existe
$ ./mayus2 no_existe 2>error  # 2> redirecciona el error estándar
$ cat error
```

Comentemos la diferencia entre '**`./mayus2 <fich`**' y '**`./mayus2 fich`**'. Como se observa el resultado es el mismo, sin embargo, la cadena **`<fich`** es interpretada por el shell, **`mayus2`** no ve como argumento el nombre de fichero `fich`, sino que pasa a mayúsculas su entrada estándar, que el shell ha redireccionado para que provenga del fichero. Por el contrario, en la segunda orden se pasa el nombre **`fich`** como argumento para **`mayus2`**, el cual lee el fichero y lo pasa a mayúsculas.

Como se dijo, existen muchas órdenes en UNIX con la estructura de **`mayus2`**, un ejemplo es **`sort`** (que como su nombre indica sirve para ordenar líneas) cuya sintaxis es:

`sort [opciones] [lista de ficheros]`

es posible pasarle a sort una lista de nombres de fichero, como en

```
$ sort fich1 fich2 fich3
```

pero si no se le pasan argumentos también se puede utilizar para ordenar la entrada estándar:

```
$ sort
ghi
abc
def
ctrl-d
abc
def
ghi
```

Otra orden muy útil es **cat**, utilizada para concatenar (con**CAT**) ficheros de texto, su sintaxis es:

$ cat [opciones] [lista de ficheros]

cat concatena los ficheros y saca dicha concatenación por la salida estándar, algunas variantes de uso de cat muy utilizadas son:

```
$ cat >nue_fich  # se utiliza para crear pequeños ficheros de texto sin utilizar un editor
$ cat fich  # muestra el contenido de fich
$ cat fich1 fich2 ... >>x  # anexa al final del fichero x los contenidos de fich1 fich2 ...
```

Operadores de redirección (redirection operators)

Operador Redirección	Efecto del Operador
>	Redirige la stdout a un fichero, si no existe lo crea, y si existe lo sobrescribe (borra el contenido antes de incluir el nuevo contenido) **ls -l > lista.txt** (La salida del comando se envía a un archivo en vez de la terminal.)
>>	Añade la stdout a un fichero, pero a diferencia del anterior la añade al fichero especificado **ps -ef >> procesos.txt** (Concatena al archivo procesos.txt la salida del comando.)
2>	Redirige la stderr a un fichero, si no existe lo crea, y si existe lo sobrescribe (borra el contenido antes de incluir el nuevo contenido)
2>>	Redirige la stderr a un fichero, pero a diferencia del anterior la añade al fichero especificado
&>	Crea un fichero tanto con stdout como con stderr , si ya existe elimina primero su contenido
<	Envía el contenido del fichero especificado como stdin
<<	Utiliza como stdin las líneas de texto pasadas
<>	El fichero especificado será utilizado tanto como stdin como stdout

5.5. Interconexiones (pipes)

Muchas veces es útil combinar la acción de varias órdenes. Por ejemplo, la orden **who** muestra en la salida estándar una línea con información sobre las sesiones abiertas en el sistema. La primera información de dicha línea es el nombre del usuario que tiene abierta la

sesión. Se puede combinar la salida de esta orden con **sort** para obtener una lista ordenada por nombre de usuario de las sesiones abiertas:

```
$ who >temp
$ sort <temp
```

Como este tipo de combinaciones son muy utilizadas UNIX proporciona una llamada al sistema para redireccionar la entrada estándar de un proceso para que en vez de provenir del terminal provenga de la salida estándar de otro proceso. Los shells hacen uso de dicha llamada al sistema, y proporcionan un metacarácter, el '|', para poder especificar la redirección. Así, si escribimos:

```
$ who | sort
```

el shell creará dos procesos, y conectará la salida estándar de **who** a la entrada estándar de **sort**. La orden **who** no es consciente de que su salida estándar ha sido redireccionada, ni la orden **sort** de que lo ha sido su entrada estándar. Entre los dos procesos se ha creado un canal de comunicación llamado pipe, esta palabra, de origen inglés, ha sido traducida principalmente como **interconexión, tubo y tubería**.

Las interconexiones sirven para combinar la acción de varias órdenes (aprovechando que la mayoría de las órdenes de UNIX utilizan su entrada y/o salida estándar) creando órdenes más potentes, he aquí varios ejemplos:

```
$ ls | wc -l
$ who | grep pepe
$ who | grep pepe | wc -l
```

la orden **wc** (*word count*) sirve para contar el número de caracteres, palabras y líneas de la entrada estándar, o de uno o varios ficheros de texto que se pasen como argumentos. Con la opción **-l** sólo cuenta el número de líneas. Como **ls** lista una línea por fichero o directorio, **ls | wc -l** mostrará el número de ficheros y directorios que cuelgan del directorio de trabajo. La orden **grep** (su nombre procede de la orden g/re/p del editor ed: buscar globalmente una expresión regular e imprimir las líneas que la contienen) selecciona de su entrada estándar (o de los ficheros dados como argumentos) las líneas que contienen un patrón de texto (en el ejemplo pepe), por lo tanto la última interconexión nos va a servir para contar el número de sesiones abiertas por el usuario **pepe**.

Por último, hay que decir que se pueden combinar en una misma orden interconexiones y redireccionamientos, como en:

```
$ ls | wc -l >num
```

Ejemplos de uso

```
$ ls | grep cap | wc -l
```

Ejercicios

1. Crea los siguientes programas y los ejecutas. ¿Cuántos mensajes de "Hola mundo" escribe el primero?, ¿cuántos mensajes "Hola mundo" escribe el segundo?, ¿esos mensajes los escribe el mismo proceso? Modifica el segundo programa para que muestre el id del proceso que escribe el mensaje y el id de proceso padre. Explica su comportamiento.

   La función **getpid()** devuelve el ID de proceso del proceso que realiza la llamada.

   La función **getppid()** devuelve el ID de proceso del proceso padre del proceso que realiza la llamada.

   programa 1

```cpp
#include <iostream>
#include <unistd.h>

int main() {

        fork();
        std::cout << "Hola mundo" << std::endl;

        return 0;
}
```

   programa 2

```cpp
#include <iostream>
#include <unistd.h>

int main() {

        fork();
        fork();
        std::cout << "Hola mundo" << std::endl;

        return 0;
}
```

2. Realice un programa en C++ equivalente a la orden *echo*, es decir, un programa que lo único que haga sea mostrar sus argumentos. Llámelo *eco*. Ejecute

```
$ ./eco a b
$ ./eco *
```

   ¿por qué en la segunda línea eco no muestra como salida un asterisco? Pruebe a realizar $ echo *. ¿Intuye para qué sirve la barra invertida?

3. Dada la orden

```
$ cat a b >x
```

¿qué valores tendrán las variables *argc* y *argv* para dicha ejecución del programa cat? (Nota: lo puede deducir empleando la orden eco, propuesta en el ejercicio anterior).

4. Suponga que en su directorio de trabajo no existe el fichero *usuarios*. Escriba mal el nombre de una orden, como en

```
$ woh >usuarios
woh: orden no encontrada
```

y compruebe mediante la orden ls si existe el fichero *usuarios*. ¿Por qué cree que existe? ¿Por qué tiene tamaño 0?

5. Dadas las órdenes *head* y *tail*, con la siguiente sintaxis y semántica:

head -n N [fich] muestra en la salida estándar las N primeras líneas de la entrada estándar, o del fichero *fich*, si se usa segundo parámetro.

tail -n N [fich] muestra en la salida estándar las N últimas líneas de la entrada estándar, o del fichero *fich*, si se usa segundo parámetro.

escriba una orden que muestre en la pantalla la tercera línea del fichero */etc/passwd*.

Preguntas frecuentes

¿Por qué al ejecutar una orden en el shell me dice "orden no encontrada" o "command not found"?

Seguramente, al ejecutar alguno de tus programas desde el shell (el terminal) te has encontrado con este problema:

```
alumno@ubuntu:~$ shell
bash: shell: orden no encontrada
alumno@ubuntu:~$
```

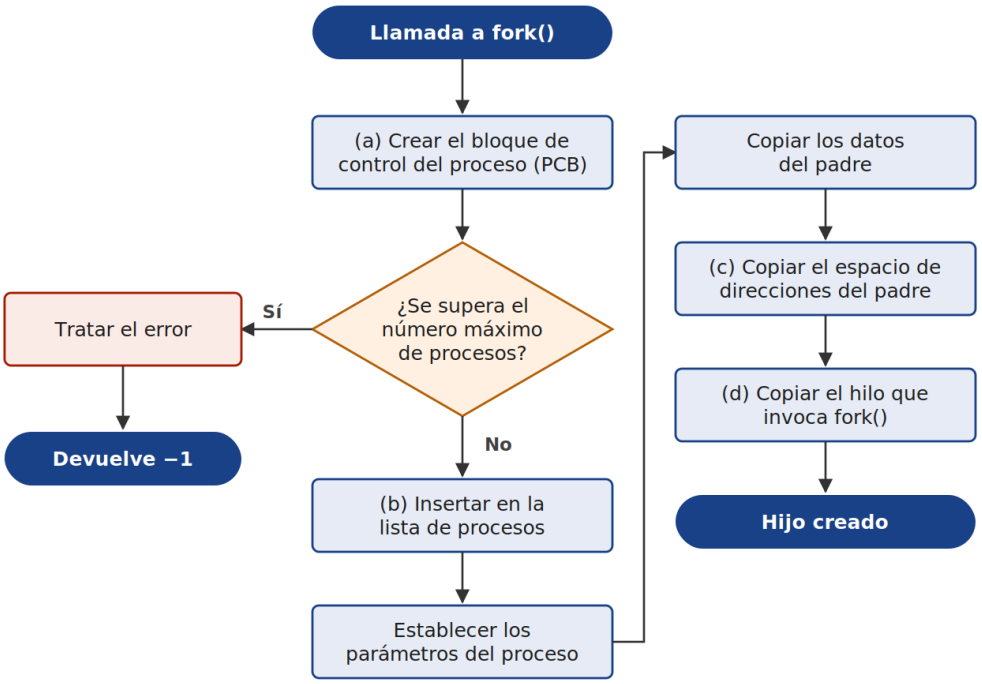
Para que el intérprete de órdenes pueda encontrar los programas (ficheros ejecutables) que queremos ejecutar, utiliza una variable llamada PATH. Esta variable PATH (ruta, trayectoria o camino) es una variable de entorno que contiene todas aquellas trayectorias donde existen ficheros ejecutables y, por tanto, donde debe buscarse el programa que queremos ejecutar. En consecuencia, si nuestro directorio de trabajo (`/home/alumno`) no está en esa variable PATH, no buscará en este camino ficheros ejecutables. Para poder ejecutar desde sitio en el que se encuentra `miprograma` debo indicarle la ruta, tengo dos opciones:

- Proporcionarle el camino absoluto, esto es, `/home/alumno/miprograma`.

- Proporcionarle un camino relativo, "decirle que busque aquí". En UNIX/Linux hay una manera de indicar aquí, el punto "`.`". El punto es equivalente a "en el lugar en el que estoy situado dentro del sistema de ficheros". Por tanto, puedo ejecutar miprograma escribiendo en el shell: `./miprograma`.

Puede consultar el contenido de esa variable con la orden echo "$PATH": verá una lista de directorios separados por dos puntos. Para saber en cuál de ellos se encuentra un programa, utilice type o which, por ejemplo type ls.

fork y wait ¿Para qué sirven las llamadas fork y wait?



Para crear nuevos procesos, el UNIX/Linux dispone únicamente de una llamada al sistema, fork, sin ningún tipo de parámetros. Su prototipo es

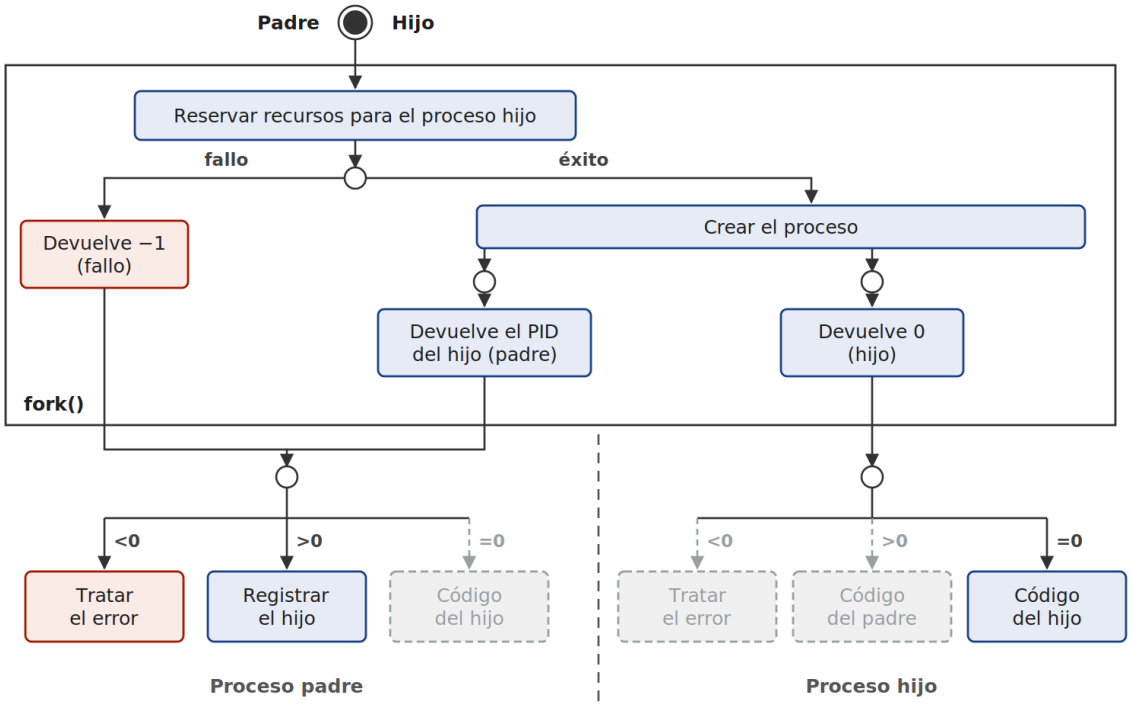
```
pid_t fork(void);    /* declarado en <unistd.h> */
```

Al llamar a esta función se crea un nuevo proceso (proceso hijo), idéntico en todo (en código, datos y Bloque de Control de Proceso) al proceso que ha realizado la llamada (proceso padre) pero en una nueva zona de memoria. Los espacios de memoria del padre y el hijo son disjuntos, por lo que el proceso hijo es una copia idéntica del padre que a partir de ese momento sigue su vida separada, sin afectar a la memoria del padre; y viceversa.

Siendo más concretos, las variables del proceso padre son inicialmente las mismas que las del hijo. Pero si cualquiera de los dos procesos altera una variable, el cambio sólo repercute en su copia local. Padre e hijo no comparten memoria.

El punto del programa donde el proceso hijo comienza su ejecución es justo en el retorno de la función `fork`, al igual que ocurre con el padre.

Si el proceso hijo fuera un mero clon del padre, ambos ejecutarían las mismas instrucciones, lo que en la mayoría de los casos no tiene mucha utilidad. El Linux permite distinguir si se es el proceso padre o el hijo por medio del valor de retorno de `fork`. **Esta función devuelve un cero al proceso hijo, y el identificador de proceso (PID) del hijo al proceso padre**. Cómo se garantiza que el PID siempre es no nulo, basta aplicar un if para determinar quién es el padre y quién el hijo para así ejecutar distinto código.



Con un pequeño ejemplo:

```
int main () {
        int x=1;
        if ( fork()==0 ) {
                std::cout << "Soy el hijo, x=" << x << std::endl;
        } else {
                x=33;
                std::cout << "Soy el padre, x=" << x << std::endl;
        }
}
```

Este programa mostrará por la salida estándar las cadenas "Soy el hijo, x=1" y "Soy el padre, x=33", en un orden que dependerá del compilador y del planificador de procesos de la máquina donde se ejecute.

Como aplicación de fork a la ejecución de programas, véase este otro pequeño ejemplo, que además nos introducirá en nuevas herramientas:

```
if ( fork()==0 ) {
        execlp ("ls","ls","-l","/usr/include",(char *)nullptr);
        std::cerr << "Si ves esto, no se pudo ejecutar el programa";
        return 1;
} else {
        int estado_hijo=0;
        wait(&estado_hijo);
}
```

Este fragmento de código lanza a ejecución la orden `ls -l /usr/include`, y espera por su terminación.

En la línea 1 se verifica si se es padre o hijo. Generalmente es el hijo el que toma la iniciativa de lanzar un fichero a ejecución, y en la línea 2 se invoca a un programa con execlp. Las líneas 3 y 4 sólo se ejecutarán si la llamada execlp no se ha podido cumplir. La llamada a return garantiza que el hijo no se dedicará a hacer más tareas.

Las líneas de la 6 y 7 forman el código que ejecutará el proceso padre, mientras el hijo anda a ejecutar sus cosas. Aparece una nueva llamada el sistema, la función `wait`. Esta función bloquea al proceso llamador hasta que alguno de sus hijos termina. Para nuestro ejemplo, dejará al padre bloqueado hasta que se ejecute el programa lanzado o se ejecute la línea 4, terminando en ambos casos el discurrir de su único hijo.

Es decir, la función `wait` es un mecanismo de sincronización entre un proceso padre y sus hijos.

La llamada `wait` recibe como parámetro un puntero a entero donde se deposita el código de estado devuelto por el proceso hijo al terminar; y retorna el PID del hijo. El PID del hijo es una información que puede ser útil cuando se han lanzado varios procesos hijos y se desea discriminar quién exactamente ha terminado. En nuestro ejemplo no hace falta este valor, porque sólo hay un hijo por el que esperar.

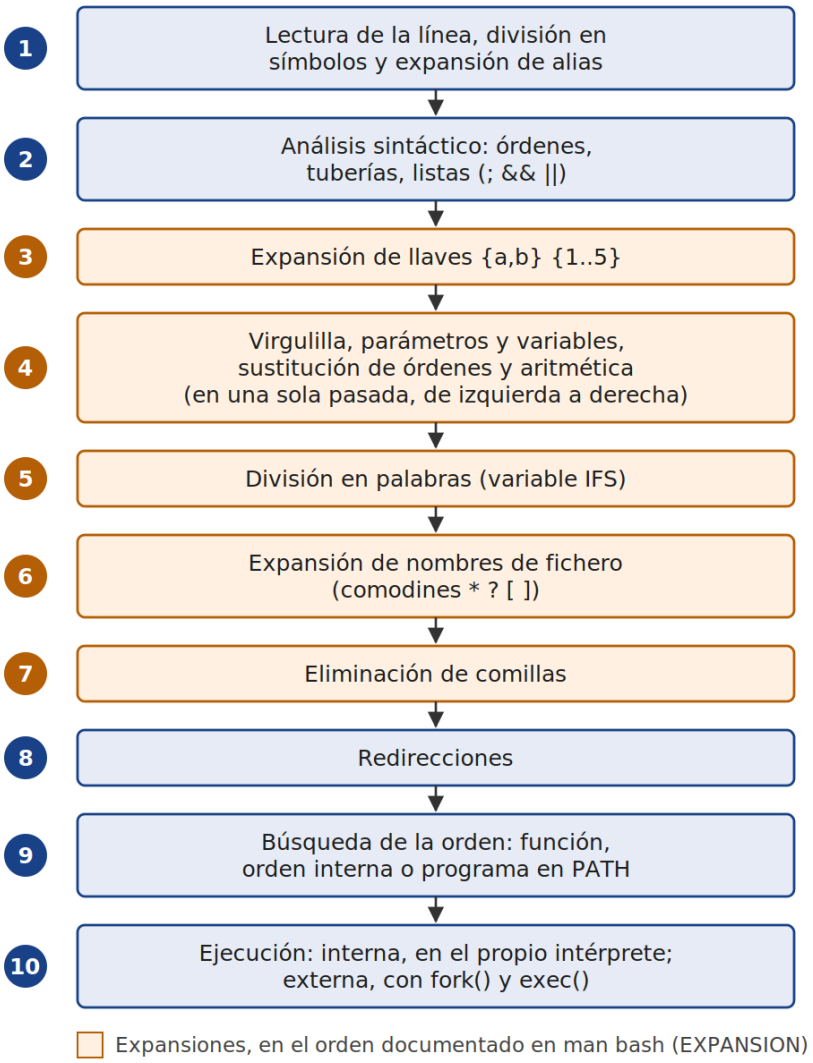
Shell ¿Cómo interpreta las órdenes el shell?

Lo primero que nos interesa saber, es el orden en que son interpretados los comandos. Partiremos de la base que se pueden ejecutar más de un comando en una línea. Esto se logra separándolos con un punto y coma (`;`).

Lo primero que el shell realiza es dividir estas órdenes o comandos en lo que se denomina "tokens". Cada token es un comando, junto con sus argumentos, a ejecutar. Una vez que el shell puede saber cuántos tokens hay, determina su sintaxis.

Si existe un error en este punto, ninguno de los tokens se ejecutará y el shell nos advertirá de un posible error de sintaxis. Si la sintaxis es correcta, comienza la interpretación de los comandos. A partir de aquí el shell trata de ver si alguno de los tokens es en realidad una función del propio shell. Si existe, esta será expandida antes de ser ejecutada. Luego, se expanden los alias. Si alguno de los tokens es en realidad un alias, ya sea creado por el

usuario o no, este se expandirá antes de ejecutarse. Si el alias llama a otro alias, este también se expandirá antes de ejecutarse. Una vez que se han expandido los alias y funciones el shell comenzará a evaluar las variables y las sustituirá con su valor. Al terminar esto, se expandirá cualquier comodín utilizado expandiéndolo al nombre de archivo o comando que concuerde de acuerdo a las reglas antedichas.



Ahora, aunque está la mayor parte del comando evaluado, el shell tendrá todavía que ver si el primero de los tokens corresponde a un comando del shell o a un programa externo. Si esto ocurriera, se tendrá que consultar la ruta de búsqueda. Luego evaluará si existe algún tipo de redirección o canalización por el cual la entrada o salida de un comando este redirigida a otro lugar. Una vez que se ha podido analizar todo el comando o conjunto de comandos a su mínima expresión, el shell efectuará una copia de sí mismo, a través de la llamada a sistema **`fork()`** para poder ejecutar el script. Esta copia es un hijo del shell y depende del shell que lo originó. Luego de esto, el shell hijo, usará una llamada a sistema denominada **`exec()`** para poder ejecutar el binario correspondiente. Aunque quien lo ejecuta es en realidad el shell hijo, hay que recordar que el padre todavía se encuentra en memoria esperando la finalización del proceso hijo.

Puede darse el caso que se desee que quien ejecute el comando o script sea el propio shell y no un shell hijo. Esto se logra con la utilización del comando "." punto. Al anteponer un punto al comando o script y dejando un espacio entre ellos, el shell entenderá que es el mismo quién debe interpretar este comando o script.

Argumentos ¿Cómo se pasan argumentos a una orden?

Habitualmente, cuando se lanza un programa a ejecución desde el shell, se le añaden parámetros o argumentos para definir exactamente qué queremos de él. Por ejemplo:

```
$ vi pepe.txt
$ ls -l /usr/include
$ cp pepe.cpp ../pipo.cpp
```

En el primer caso se invoca al editor vi especificando que se desea trabajar con el fichero "**`pepe.txt`**". El fichero es un parámetro pasado al shell. En el segundo caso se ejecuta la orden **`ls`** que incorpora dos parámetros, como ocurre en el último ejemplo.

¿Qué son los parámetros o argumentos? En principio, puede afirmarse que son conjuntos de caracteres separados por blancos (espacios o tabuladores). Ahora bien, no se consideran parámetros los redireccionamientos, y caracteres como el ampersand "**`&`**" o el punto y coma "**`;`**" actúan de separadores (en general, eso ocurre con todos los caracteres que tengan un significado para el intérprete de órdenes).

Ejemplo: En la orden

```
$ ls -l /usr/include &>pepe.txt
```

los parámetros son "**`ls`**", "**`-l`**" y "**`/usr/include`**" ('**`&`**' y la redirección no cuentan).

Cada programa recibe los parámetros a través de su punto de entrada, que en el caso del lenguaje C es la función **main**. El formato mínimo que acepta esta función en UNIX/Linux es:

```
int main(int argc, char *argv[])
```

donde **`argc`** expresa cuántos parámetros se han reconocido, y **`argv`** es un vector de cadenas de caracteres que precisamente contienen los parámetros, siendo **`argv[i]`** el parámetro **`i`**.

Los parámetros se empiezan a numerar en 0. El parámetro 0 es el nombre del programa invocado tal y como se pasó en la línea de órdenes. En el ejemplo de **`vi pepe.txt`**, los valores de argc y argv serían:

```
argc = 2

argv[0] = "vi"

argv[1] = "pepe.txt"
```